Department of Electronics and Telecommunication Engineering

ANALYSIS REPORT

ETHERNET COMMUNICATION INVESTIGATION AND TRAFFIC ANALYSIS USING LINUX

Submitted by

Mrudula Bhandurge

Roll Number: BE203

Academic Year: 2026–27

1. Aim

To investigate Ethernet network interfaces in Linux and capture and analyse ARP, ICMP, TCP and UDP traffic using Wireshark, TShark and tcpdump.

2. Software and Environment

Component	Details
Operating System	Ubuntu 22.04.5 LTS running through WSL2 on Windows 11
Kernel	6.6.123.2-microsoft-standard-WSL2+
Wireshark	Version 3.6.2
TShark	Version 3.6.2

tcpdump	Version 4.99.1
Network tools	iproute2 and net-tools
Network interface	eth0 (virtual Ethernet interface)

3. Network Interface Investigation

The Linux network interface and its configuration were investigated using ip link show, ip addr, hostname -I and ip -s link show eth0.

Parameter	Observed value
Interface	eth0
Interface state	UP, LOWER_UP
MAC address	00:15:5d:9b:50:c1
IPv4 address	172.22.47.152/20
Broadcast address	172.22.47.255
MTU	1500 bytes
RX packets	50,746
RX errors / dropped	0 / 0
TX packets	15,181
TX errors / dropped	0 / 0

The eth0 interface was operational. No receive or transmit errors or dropped packets were reported in the observed statistics. Since the work was performed inside WSL2, eth0 represents a virtual Ethernet interface.

4. Linux Networking Command Analysis

4.1 Routing Table

The ip route command displayed the routing table. The default gateway was 172.22.32.1 through eth0. The connected network was 172.22.32.0/20, with source address 172.22.47.152.

4.2 ARP Cache

The arp -a command displayed the neighbour mapping for the gateway 172.22.32.1. Its observed MAC address was 00:15:5d:da:82:d1.

4.3 Connectivity Test

The command ping -c 4 8.8.8.8 was used to test IP connectivity. Four packets were transmitted and four were received, resulting in 0% packet loss. The observed average round-trip time was approximately 499.007 ms.

4.4 Socket Information

The ss -tuln and netstat -tuln commands were used to inspect listening TCP and UDP sockets. DNS port 53 listeners were observed at 127.0.0.53 and 10.255.255.254. UDP port 323 was also observed on localhost IPv4 and IPv6.

5. ARP Traffic Capture and Analysis

ARP traffic was captured on the eth0 interface and saved as wireshark/arp_capture.pcapng. The capture contained four ARP frames, showing two request and reply exchanges.

Frame	Observed packet	Meaning
1	00:15:5d:9b:50:c1 to Broadcast: Who has 172.22.32.1?	ARP Request from the local interface.
2	00:15:5d:da:82:d1 to 00:15:5d:9b:50:c1: 172.22.32.1 is at 00:15:5d:da:82:d1.	ARP Reply from the gateway.
3	Gateway asks who has 172.22.47.152.	ARP Request for the local

		interface.
4	Local interface replies with 00:15:5d:9b:50:c1.	ARP Reply providing the MAC address.

Observation: The capture demonstrates how ARP resolves IPv4 addresses into MAC addresses on a local network. The first request was broadcast, and the gateway returned its MAC address.

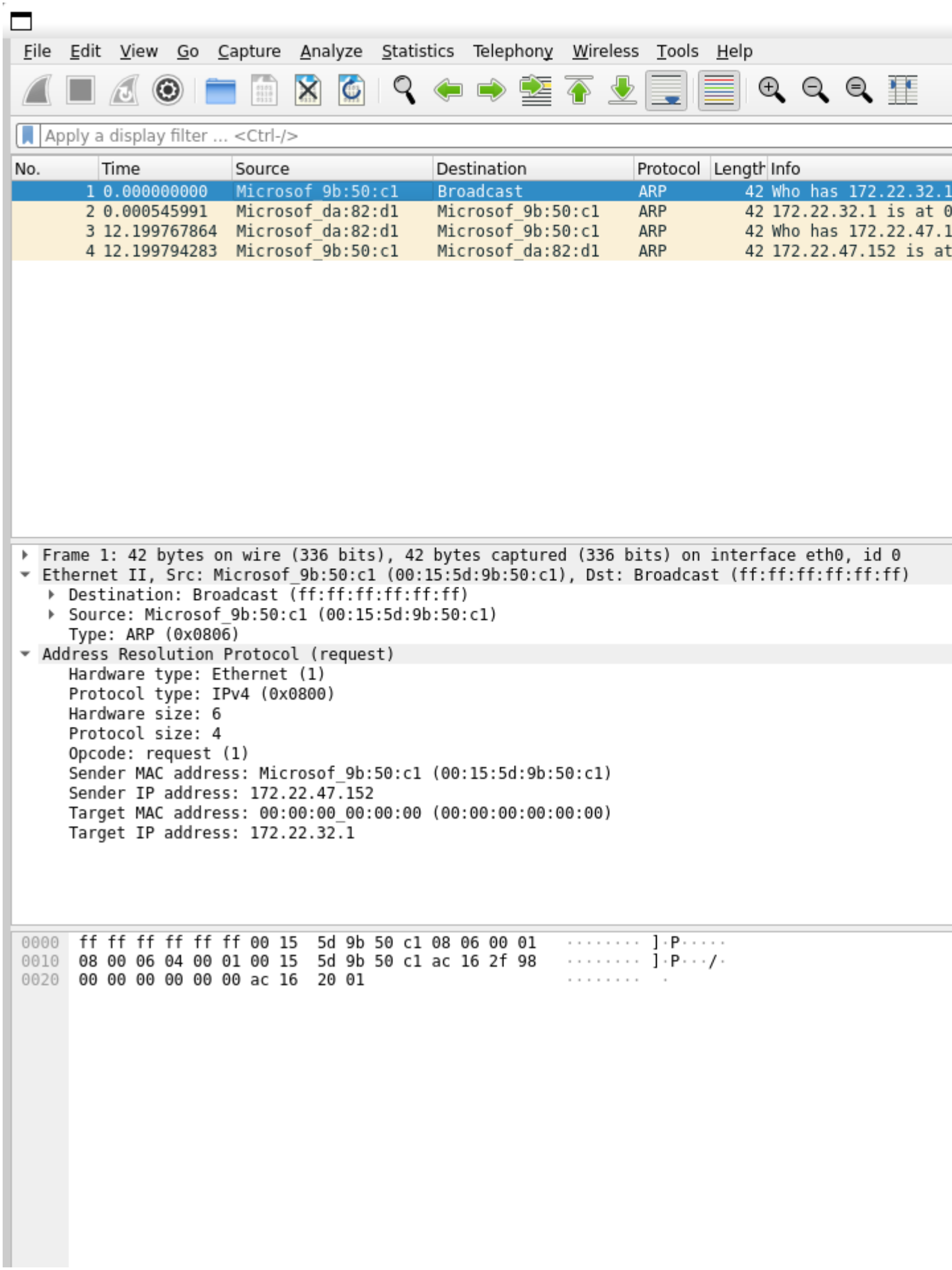

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 Microsof_9b:50:c1 Broadcast ARP 42 Who has 172.22.32.1
2 0.000545991 Microsof_da:82:d1 Microsof_9b:50:c1 ARP 42 172.22.32.1 is at 0
3 12.199767864 Microsof_da:82:d1 Microsof_9b:50:c1 ARP 42 Who has 172.22.47.1
4 12.199794283 Microsof_9b:50:c1 Microsof_da:82:d1 ARP 42 172.22.47.152 is at
Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
Ethernet II, Src: Microsof_9b:50:c1 (00:15:5d:9b:50:c1), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Destination: Broadcast (ff:ff:ff:ff:ff:ff)
Source: Microsof_9b:50:c1 (00:15:5d:9b:50:c1)
Type: ARP (0x0806)
Address Resolution Protocol (request)
Hardware type: Ethernet (1)
Protocol type: IPv4 (0x0800)
Hardware size: 6
Protocol size: 4
Opcode: request (1)
Sender MAC address: Microsof_9b:50:c1 (00:15:5d:9b:50:c1)
Sender IP address: 172.22.47.152
Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
Target IP address: 172.22.32.1
0000 ff ff ff ff ff ff 00 15 5d 9b 50 c1 08 06 00 01
0010 08 00 06 04 00 01 00 15 5d 9b 50 c1 ac 16 2f 98
0020 00 00 00 00 00 00 ac 16 20 01

Figure 1: ARP capture and packet analysis in Wireshark.

6. ICMP Traffic Capture and Analysis

ICMP traffic was generated using ping -c 4 8.8.8.8 and saved as wireshark/icmp_capture.pcapng. The capture contained eight packets: four Echo Requests and four Echo Replies.

Parameter	Observed value
Source IP of request	172.22.47.152
Destination IP of request	8.8.8.8
Echo Request type	8, code 0
Echo Reply type	0, code 0
Request TTL	64
Reply TTL	116
Packet loss	0%
Average round-trip time in this capture test	452.654 ms

Observation: Echo Request packets travelled from the local interface to 8.8.8.8, and Echo Reply packets returned from the destination. This confirmed successful ICMP communication during the test.

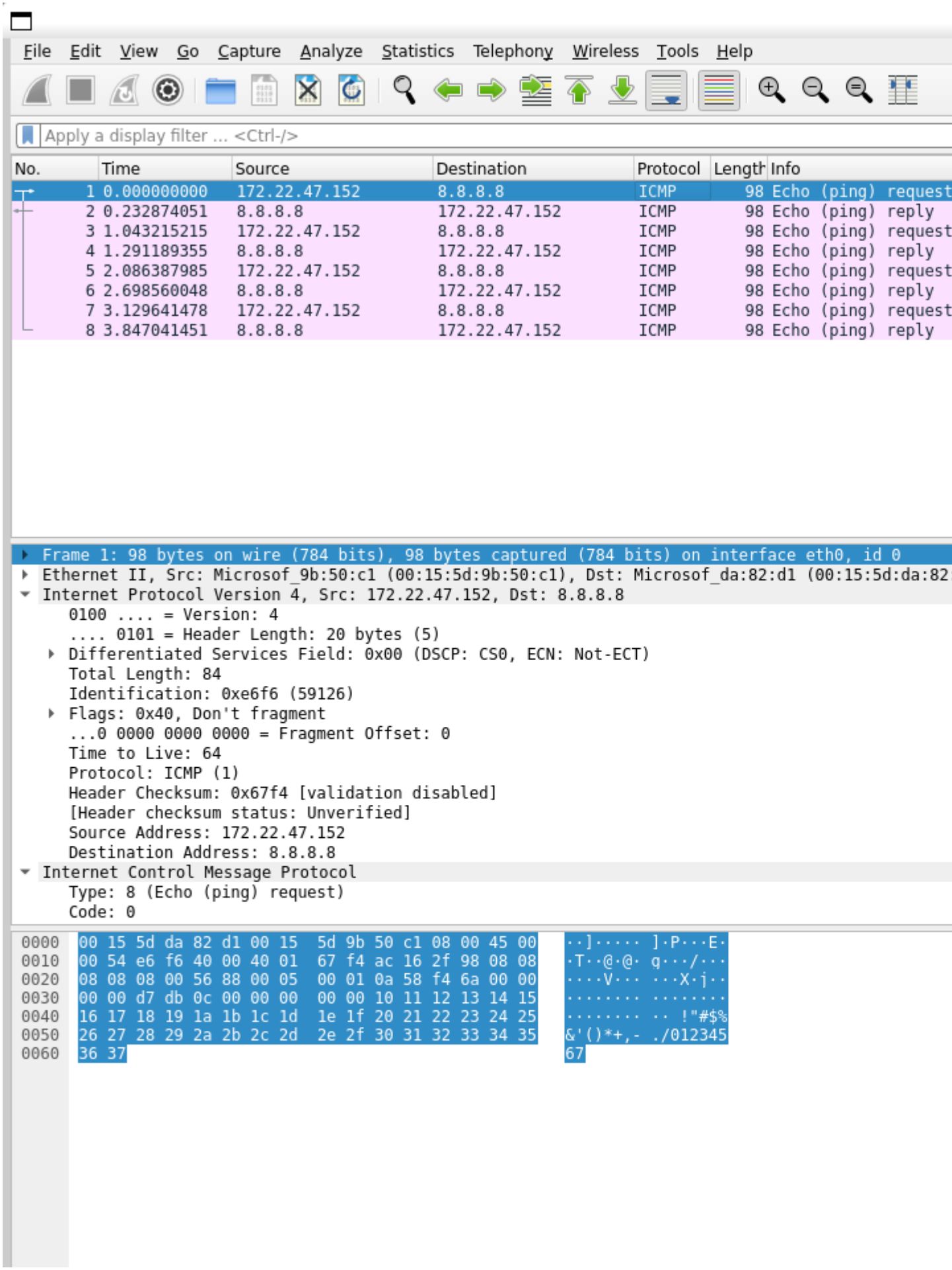

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 172.22.47.152 8.8.8.8 ICMP 98 Echo (ping) request
2 0.232874051 8.8.8.8 172.22.47.152 ICMP 98 Echo (ping) reply
3 1.043215215 172.22.47.152 8.8.8.8 ICMP 98 Echo (ping) request
4 1.291189355 8.8.8.8 172.22.47.152 ICMP 98 Echo (ping) reply
5 2.086387985 172.22.47.152 8.8.8.8 ICMP 98 Echo (ping) request
6 2.698560048 8.8.8.8 172.22.47.152 ICMP 98 Echo (ping) reply
7 3.129641478 172.22.47.152 8.8.8.8 ICMP 98 Echo (ping) request
8 3.847041451 8.8.8.8 172.22.47.152 ICMP 98 Echo (ping) reply
Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
Ethernet II, Src: Microsof_9b:50:c1 (00:15:5d:9b:50:c1), Dst: Microsof_da:82:d1 (00:15:5d:da:82
Internet Protocol Version 4, Src: 172.22.47.152, Dst: 8.8.8.8
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 84
Identification: 0xe6f6 (59126)
Flags: 0x40, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: ICMP (1)
Header Checksum: 0x67f4 [validation disabled]
[Header checksum status: Unverified]
Source Address: 172.22.47.152
Destination Address: 8.8.8.8
Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
0000 00 15 5d da 82 d1 00 15 5d 9b 50 c1 08 00 45 00
0010 00 54 e6 f6 40 00 40 01 67 f4 ac 16 2f 98 08 08
0020 08 08 08 00 56 88 00 05 00 01 0a 58 f4 6a 00 00
0030 00 00 d7 db 0c 00 00 00 00 00 10 11 12 13 14 15
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35
0060 36 37

Figure 2: ICMP Echo Request and Echo Reply analysis in Wireshark.

7. TCP Traffic Capture and Analysis

TCP traffic was generated by accessing http://example.com/ using curl with proxy bypass enabled. The capture was saved as wireshark/tcp_capture.pcapng and contained 12 packets.

Parameter	Observed value
Client IP	172.22.47.152
Client port	38592
Server IP	104.20.23.154
Server port	80
Application protocol	HTTP/1.1
HTTP response	200 OK
Total captured packets	12

The first three packets showed the TCP three-way handshake: SYN, SYN-ACK and ACK. The client then sent an HTTP GET request. The server returned an HTTP/1.1 200 OK response. The final packets showed connection termination using FIN-ACK and ACK messages.

Observation: The capture demonstrated TCP connection establishment, HTTP request-response communication and connection termination.

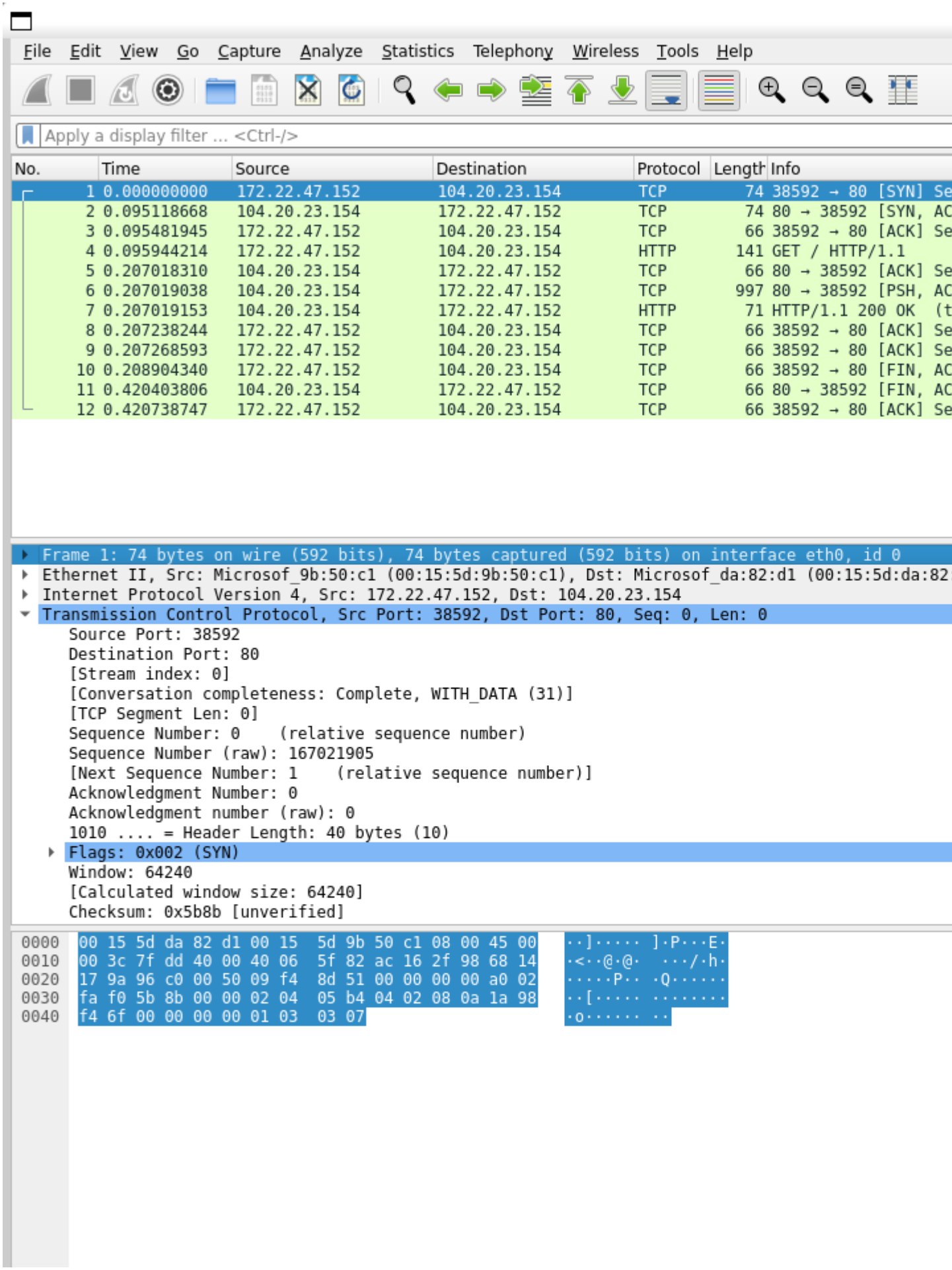

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 172.22.47.152 104.20.23.154 TCP 74 38592 → 80 [SYN] Se
2 0.095118668 104.20.23.154 172.22.47.152 TCP 74 80 → 38592 [SYN, AC
3 0.095481945 172.22.47.152 104.20.23.154 TCP 66 38592 → 80 [ACK] Se
4 0.095944214 172.22.47.152 104.20.23.154 HTTP 141 GET / HTTP/1.1
5 0.207018310 104.20.23.154 172.22.47.152 TCP 66 80 → 38592 [ACK] Se
6 0.207019038 104.20.23.154 172.22.47.152 TCP 997 80 → 38592 [PSH, AC
7 0.207019153 104.20.23.154 172.22.47.152 HTTP 71 HTTP/1.1 200 OK (t
8 0.207238244 172.22.47.152 104.20.23.154 TCP 66 38592 → 80 [ACK] Se
9 0.207268593 172.22.47.152 104.20.23.154 TCP 66 38592 → 80 [ACK] Se
10 0.208904340 172.22.47.152 104.20.23.154 TCP 66 38592 → 80 [FIN, AC
11 0.420403806 104.20.23.154 172.22.47.152 TCP 66 80 → 38592 [FIN, AC
12 0.420738747 172.22.47.152 104.20.23.154 TCP 66 38592 → 80 [ACK] Se
Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
Ethernet II, Src: Microsof_9b:50:c1 (00:15:5d:9b:50:c1), Dst: Microsof_da:82:d1 (00:15:5d:da:82
Internet Protocol Version 4, Src: 172.22.47.152, Dst: 104.20.23.154
Transmission Control Protocol, Src Port: 38592, Dst Port: 80, Seq: 0, Len: 0
Source Port: 38592
Destination Port: 80
[Stream index: 0]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 167021905
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1010 .... = Header Length: 40 bytes (10)
Flags: 0x002 (SYN)
Window: 64240
[Calculated window size: 64240]
Checksum: 0x5b8b [unverified]
0000 00 15 5d da 82 d1 00 15 5d 9b 50 c1 08 00 45 00
0010 00 3c 7f dd 40 00 40 06 5f 82 ac 16 2f 98 68 14
0020 17 9a 96 c0 00 50 09 f4 8d 51 00 00 00 00 a0 02
0030 fa f0 5b 8b 00 00 02 04 05 b4 04 02 08 0a 1a 98
0040 f4 6f 00 00 00 00 01 03 03 07

Figure 3: TCP handshake, HTTP communication and connection termination in Wireshark.

8. UDP Traffic Capture and Analysis

UDP traffic was generated by sending a DNS query for example.com to 8.8.8.8 on UDP port 53. The capture was saved as wireshark/udp_capture.pcapng and contained two packets.

Parameter	Observed value
Client IP	172.22.47.152
DNS server IP	8.8.8.8
Client UDP port	33131
Server UDP port	53
DNS query	example.com
Query UDP length	37 bytes
Response UDP length	69 bytes
Total captured packets	2

The first packet was a DNS query from the client to the DNS server. The second packet was the DNS response. The response contained the IPv4 addresses 104.20.23.154 and 172.66.147.243 for example.com.

Observation: The capture demonstrated DNS request-response communication using UDP port 53.

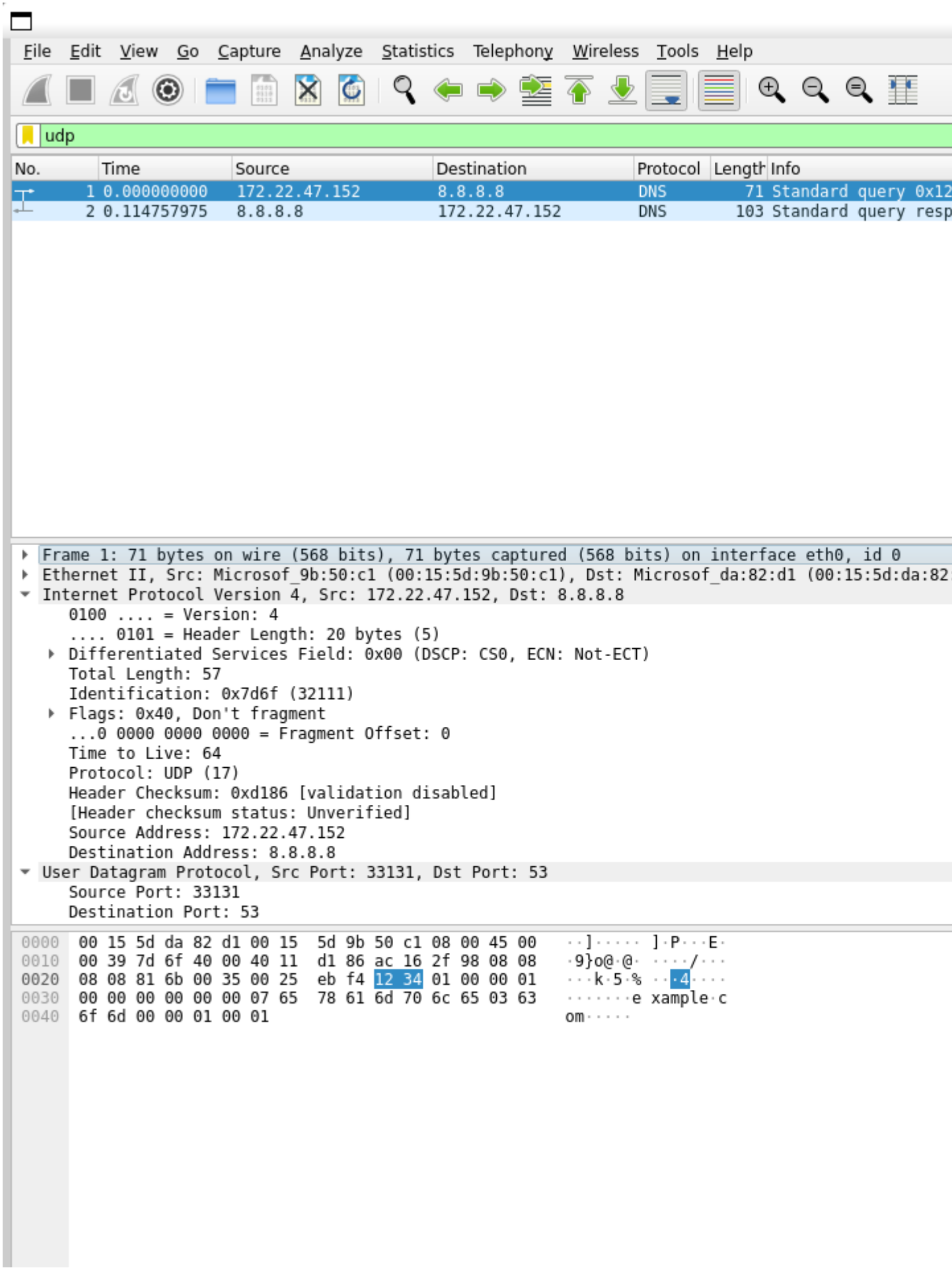

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
udp
No. Time Source Destination Protocol Length Info
1 0.000000000 172.22.47.152 8.8.8.8 DNS 71 Standard query 0x12
2 0.114757975 8.8.8.8 172.22.47.152 DNS 103 Standard query resp
Frame 1: 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface eth0, id 0
Ethernet II, Src: Microsof_9b:50:c1 (00:15:5d:9b:50:c1), Dst: Microsof_da:82:d1 (00:15:5d:da:82
Internet Protocol Version 4, Src: 172.22.47.152, Dst: 8.8.8.8
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 57
Identification: 0x7d6f (32111)
Flags: 0x40, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: UDP (17)
Header Checksum: 0xd186 [validation disabled]
[Header checksum status: Unverified]
Source Address: 172.22.47.152
Destination Address: 8.8.8.8
User Datagram Protocol, Src Port: 33131, Dst Port: 53
Source Port: 33131
Destination Port: 53
0000 00 15 5d da 82 d1 00 15 5d 9b 50 c1 08 00 45 00
0010 00 39 7d 6f 40 00 40 11 d1 86 ac 16 2f 98 08 08
0020 08 08 81 6b 00 35 00 25 eb f4 12 34 01 00 00 01
0030 00 00 00 00 00 00 07 65 78 61 6d 70 6c 65 03 63
0040 6f 6d 00 00 01 00 01

Figure 4: UDP DNS query and response analysis in Wireshark.

9. Packet Capture Validation Using TShark

All four capture files were checked using TShark protocol hierarchy statistics. The observed frame counts and protocol hierarchy were:

Capture file	Frames	Observed protocol hierarchy
arp_capture.pcapng	4	Ethernet → ARP
icmp_capture.pcapng	8	Ethernet → IPv4 → ICMP
tcp_capture.pcapng	12	Ethernet → IPv4 → TCP → HTTP
udp_capture.pcapng	2	Ethernet → IPv4 → UDP → DNS

The protocol hierarchy confirmed that each capture contained the expected protocol. All four capture files were readable by TShark.

10. Result

Linux network interface investigation and networking command analysis were completed successfully. ARP, ICMP, TCP and UDP traffic were captured and analysed. The four capture files were validated using TShark, and corresponding Wireshark screenshots were collected.

11. Conclusion

The experiment provided practical understanding of Ethernet-based communication and common network protocols. Interface inspection and Linux networking commands helped examine the network configuration. Packet captures demonstrated ARP address resolution, ICMP connectivity testing, TCP connection establishment and HTTP communication, and UDP-based DNS query-response communication. The analysis improved understanding of how network traffic can be captured, inspected and interpreted using Linux networking tools and Wireshark.